Register Number: 192425207

Name: Anand Paul. D

CO5– AT1

MLA0201 – Fundamentals of Machine Learning

Question 1: Reinforcement Learning for Warehouse Robot Navigation — 20 Marks

1. Problem Formulation

The warehouse navigation problem can be formulated as a **Markov Decision Process (MDP)**.

- **State (S):** Robot's current position, battery level, and number of moves used.
- **Actions (A):** Up, Down, Left, Right.
- **Transition:** The robot moves to a new position after an action; because the environment is stochastic, the intended movement may sometimes fail.
- **Reward (R):**
  - Reaching the destination → **large positive reward**.
  - Normal movement → **small negative reward** to reduce travel time and battery usage.
  - Moving toward a blocked path → **large negative penalty**.
- **Goal:** Reach the destination within **25 moves** while conserving the battery.

2. Exploration Strategy

I would use **ε-greedy exploration**.

- With probability **ε**, the robot selects a random action for exploration.
- With probability **1 − ε**, it selects the action with the highest estimated value.
- Initially, ε can be higher so that the robot explores different paths.
- ε can gradually decrease as the robot learns the warehouse environment.
- This provides a balance between **exploration and exploitation**.

3. Value Iteration

Value Iteration can be used to calculate the optimal value of every state.

The Bellman update is:

$$V(s) = \max_a \sum_{s'} P\,(s', \mid sa)[R(s, a, s') + \gamma V(s')]$$

Where:

- $V(s)$= value of current state.
- a= possible movement.
- $P(s', | sa)$= probability of reaching next state.
- R= reward or penalty.
- γ= discount factor.

The process is:

1. Initialize the value of all states.
2. Consider all four possible movements.
3. Calculate the expected value of each action.
4. Select the action with the maximum expected value.
5. Repeat until the values converge.
6. Generate the optimal policy from the final values.
7. Stop an episode if the robot reaches the destination, exceeds **25 moves**, or runs out of battery.

4. Optimal Policy

The resulting policy gives the best movement for each warehouse position.

For example:

Start → Right → Right → Down → Down → Left → Destination

The exact path depends on the obstacle locations and reward values.

5. Justification

Value Iteration is suitable because:

- It considers the **stochastic nature** of the environment.
- It can find a policy that maximizes expected reward.
- Negative movement rewards encourage **shorter paths**.
- Battery penalties encourage **energy-efficient navigation**.
- Obstacle penalties prevent the robot from choosing blocked paths.
- The 25-move limit can be included in the state.
- The 30-unit battery constraint can also be included in the state.

Conclusion

Therefore, **ε-greedy exploration + Value Iteration** provides a suitable RL solution. The robot explores different routes initially and gradually learns an optimal policy that reaches the destination while avoiding obstacles, minimizing movement, and conserving battery.

Question 2: Multi-Armed Bandit for Online Advertisement Selection — 20 Marks

1. K-Armed Bandit Formulation

This problem can be formulated as a **5-Armed Bandit problem** because there are five advertisements.

- **K = 5**
- **Arms:** Advertisement 1, Advertisement 2, Advertisement 3, Advertisement 4, Advertisement 5.
- **Action:** Select one advertisement for each visitor.
- **Reward:** 1 if the visitor clicks the advertisement and 0 if there is no click.
- **Click probability:** Initially unknown for all five advertisements.
- **Horizon:** 10,000 user interactions.
- **Objective:** Maximize total clicks and minimize cumulative regret.

For advertisement i:

$$R_i = \begin{cases} 1 & \text{if user clicks} \\ 0 & \text{otherwise} \end{cases}$$

The estimated click-through rate is:

$$CTR_i = \frac{\text{Number of clicks}}{\text{Number of displays}}$$

2. Selected Strategy: UCB

I would use the **Upper Confidence Bound (UCB)** algorithm.

The UCB value is:

$$UCB_i = \bar{X}_i + \sqrt{\frac{2 \ln t}{n_i}}$$

Where:

- $\bar{X}_i$= average reward of advertisement i.
- t= total number of interactions.
- n_i= number of times advertisement $i$has been displayed.

The advertisement with the highest UCB value is selected.

3. Working of UCB

1. Initially display each advertisement at least once.
2. Calculate the average reward for every advertisement.
3. Calculate the UCB value.
4. Select the advertisement with the highest UCB.
5. Observe whether the user clicks.
6. Update the reward statistics.
7. Repeat for all **10,000 interactions**.

4. Exploration and Exploitation

UCB automatically balances exploration and exploitation.

- **Exploration:** Advertisements with fewer observations get a larger confidence bonus.
- **Exploitation:** Advertisements with higher observed CTR get higher average reward.
- As more data is collected, the confidence term decreases.
- The algorithm increasingly selects advertisements that are likely to produce more clicks.

5. Cumulative Regret

Regret represents the loss caused by not always selecting the best advertisement.

$$Regret(T) = T\mu^* - \sum_{t=1}^{T} r_t$$

Where:

- $T = 10{,}000$
- μ^*= true click probability of the best advertisement.
- r_t= reward received at interaction t.

UCB reduces cumulative regret by quickly identifying advertisements with high click probability while still exploring uncertain advertisements.

6. Effect of Reward Mechanism

The reward directly influences future advertisement selection.

- **Click → reward = 1:** Increases the estimated CTR.
- **No click → reward = 0:** Lowers the estimated CTR.
- Advertisements receiving more clicks become more likely to be selected.
- Advertisements with limited data continue receiving some exploration opportunities.

Conclusion

Therefore, the **5-armed UCB algorithm** is suitable because it does not require prior knowledge of click probabilities, balances exploration and exploitation automatically, works within the 10,000-interaction budget, and aims to maximize clicks while minimizing cumulative regret.

Question 3: Sampling Strategy for Machine Learning Model Development — 20 Marks

1. Recommended Sampling Method

I recommend **Stratified Sampling**.

The dataset contains **1 million patient records**, and the dataset may contain different patient categories such as disease-positive and disease-negative groups, age groups, gender groups, or other relevant categories.

In stratified sampling:

1. Divide the population into meaningful **strata/categories**.
2. Determine the proportion of each category.
3. Randomly select samples from every category.
4. Combine the selected samples to create the training dataset.
5. Train and evaluate the model using the representative sample.

2. Why Stratified Sampling?

Stratified Sampling is better than Simple Random Sampling for this scenario because:

- It ensures important patient categories are represented.
- It reduces sampling bias.
- It is useful when some categories are much smaller than others.
- It requires much less computation than using all 1 million records.
- It can maintain a representative distribution of the population.

- It helps improve model generalization.

For example, if the original dataset contains 90% disease-negative and 10% disease-positive patients, stratified sampling can maintain approximately the same distribution in the training sample.

3. Sample Complexity

Sample complexity describes how many training examples are required for a model to achieve a desired accuracy and confidence.

Generally:

$$m = O\left(\frac{1}{\epsilon}\left(d\log\frac{1}{\epsilon} + \log\frac{1}{\delta}\right)\right)$$

Where:

- m= required sample size.
- ϵ= allowed error.
- d= model complexity/VC dimension.
- δ= allowed probability of failure.

For **95% confidence**:

$$1 - \delta = 0.95$$

Therefore:

$$\delta = 0.05$$

The sample should be large enough to provide the required generalization performance rather than simply choosing a very small sample.

4. VC Dimension

VC dimension measures the capacity or complexity of a learning model.

- Low VC dimension → simpler model and fewer samples may be required.
- High VC dimension → more samples are required to generalize accurately.
- A model with unnecessarily high complexity may overfit the sampled data.
- Selecting an appropriate model helps reduce computational cost and improves generalization.

Therefore, the model complexity should be controlled according to the available 8 GB RAM and required accuracy.

5. Occam's Learning Principle

Occam's Learning Principle states that when multiple models explain the data, the simpler model is generally preferred.

For this problem:

- Avoid unnecessarily complex models.
- Use a simpler model that can achieve the required accuracy.
- Simpler models need less memory and computation.
- They are less likely to overfit the sample.
- This improves generalization to unseen patient records.

6. Accuracy-Confidence Boosting

The target is **above 95% prediction accuracy** with at least **95% confidence**.

This can be improved by:

- Using a sufficiently large stratified sample.
- Maintaining representation of all important patient categories.
- Using cross-validation.
- Evaluating the model on a separate test set.
- Increasing sample size if validation accuracy or confidence is insufficient.
- Avoiding overfitting through suitable model complexity.

7. Comparison of Sampling Methods

Method	**Advantage**	**Limitation**
Simple Random Sampling	Easy and unbiased when population is well mixed	May miss small patient categories
Stratified Sampling	**Represents every important category**	Requires identifying suitable strata
Monte Carlo Sampling	Useful for repeated random simulations	Not specifically designed to preserve population categories

8. Suitability for 8 GB RAM

Instead of training on all **1 million records**, a carefully selected stratified sample can be used.

The sample size should be determined using:

- Required accuracy.
- 95% confidence.
- Model complexity/VC dimension.
- Available memory.
- Validation performance.

If the initial sample does not achieve the required performance, the sample can be increased gradually.

Conclusion

Therefore, **Stratified Sampling** is the most appropriate method. It maintains representation of all important patient categories, reduces sampling bias, lowers computational cost, and supports better generalization. By selecting a sufficient sample based on **sample complexity**, controlling **VC dimension**, applying **Occam's principle**, and validating accuracy with **95% confidence**, the healthcare model can satisfy the given constraints.